

STADI PRINCIPALI

La visualizzazione di un modello grafico passa attraverso 4 stadi:

1) MODELLAZIONE 2) Elaborazione grafica 3) Rasterizzazione 4) Display

Essi costituiscono la PIPELINE GRAFICA

Come risultato si ha un insieme di vertici che specificano la scena

Compito dell'ELABORAZIONE GEOMETRICA è di determinare oggetti visibili e consistente colore. In questa fase si applicano: NORMALIZZAZIONE, CLIPPING, REMOZIONE DELLE LINEE NASCOSTE E ORDINEGGIATURA. Effettuata la proiezione si lavora con oggetti bidimensionali sottoponendoli a RASTERIZZAZIONE o SCAN CONVERSION per ottenere la proiezione della scena 3D nel piano 2D.

NOTA: il processo di trasferimento dell'immagine dal FRAME-BUFFER allo schermo è automatico

La produzione delle immagini è perseguita generalmente con due approcci:

① IMAGE-ORIENTED

La variabile fondamentale è il pixel (si assegna un colore ad ogni pixel).

Bisogna determinare quali primitive geometriche coinvolgono il pixel.

A ogni passo si rielaborano tutte le primitive

② OBJECT-ORIENTED

La variabile fondamentale è l'oggetto. È basato su un ciclo di renderizzazione per ogni oggetto. I vertici vengono processati da una serie di funzioni che ne determinano: trasformazione, ombreggiatura, visibilità. Ogni primitiva geometrica può influenzare ciascun pixel del frame buffer. (È la più utilizzata)

RASTERIZZAZIONE

Nei sistemi di grafica raster le primitive vengono approssimate da matrici di pixel. Più è densa la matrice più è precisa la renderizzazione.

La procedura con cui le figure continue si rappresentano con insieme di pixel è detta RASTERIZZAZIONE o SCAN CONVERSION

I display raster invocano gli algoritmi ogni volta che l'immagine viene modificata

Il comportamento di un materiale semitrasparente può essere modellato definendo il suo coefficiente di fresnel (% di radiazione riflessa)

Per modellare l'angolo tra la direzione d'ingresso e di uscita è definita dalla legge di Snell.

SCAN CONVERSION DEI SEGMENTI

Supponendo frame buffer, sia una matrice $n \times m$ con Origine nell'angolo in basso a sx
i pixel possono essere accessi tramite: `writepixel(int x, int y, int color)`
Il FB è totalmente discreto dunque si può lavorare con le coordinate.

DDA

Si suppone di avere un segmento definito da (x_1, y_1) e (x_2, y_2) con $m = \frac{\Delta y}{\Delta x}$ pendenza

Il più semplice algoritmo consiste nel calcolare m , incrementare x di 1 e
nel calcolare $y_i = mx_i + b$ (b ordinata del punto dove la retta incrocia $Y \forall x_i$)

Per eliminare l'inefficienza delle somme e delle moltiplicazioni in virgola mobile,
si applicano alcuni stratagemmi. Per eliminare le moltiplicazioni si usa
una tecnica incrementale che calcola un punto della retta in base a quello prec.
tracciando un pixel per ciascun k compreso tra x_1 e x_2 tramite `writepixel()`;
Per ogni variazione di x pari a $\Delta x \Rightarrow \Delta y = m * \Delta x$. Muovendosi da x_1 a x_2 ,
si incrementa x di 1 e ogni cido ossia $\Delta x = 1$ e $\Delta y = m$

Per ogni x DDA cerca il miglior y e per pendenze > 1 si prende qualche pixel.
Per pendenze > 1 si scambiano x e y . (Per questo $0 \leq m \leq 1$)

ALGORITMO DDA

```
{ int x; float dy, dx, y, m;  
  dy = y2 - y1; dx = x2 - x1;  
  m = dy / dx;  
  y = y1  
  for (x = x1; x <= x2; x++)  
  { writepixel(x, round(y), line-color);  
    y += m;  
  }  
}
```

BRESENHAM

Ha sostituito il DDA poiché evita i calcoli in virgole mobile.

Condizioni uguali al DDA. Supponiamo di essere nella situazione di $(i, 5)$ acceso.

Per $x=i \rightarrow y=mx+b$ deve passare da $(i, 5)$. Passando a $x=i+1$, mi indico che bisogna accendere o $(i+1, 5)$ o $(i+1, 5+1)$. Riformuliamo il problema in termini di una var. decisionale $d = b - a$ con a e b che rappresentano le distanze tra la retta e i due pixel candidati. Se $d < 0$ si accende il pixel più in basso, altrimenti in alto.

Per calcolare si usa una tecnica incrementale che elimina i calcoli in virgola mobile.

Ci sono due situazioni che dipendono dal fatto che sia stata incrementata y .

$d_i \leq 0$ $d(i+1) = d(i) + 2\Delta y$ $d_i > 0$ $d(i+1) = d(i) + 2(\Delta y - \Delta x)$. In sintesi:

- 1) A ogni passo l'algoritmo sceglie tra due pixel in base al segno di d
- 2) d dipende dal valore d del passo precedente (non richiede virgole mobile)
- 3) è più efficiente del DDA e rappresenta l'algoritmo di riferimento

d è inizializzato come $d = 2(y_2 - y_1) - (x_2 - x_1)$
 $\swarrow$ $d < 0$ angolo $< 22,5^\circ$ scelto a est
 $\searrow$ $d > 0$ angolo $> 22,5^\circ$ scelto a nord est

ALGORITMO

$\{ dx = x_2 - x_1; dy = y_2 - y_1; m = dy/dx;$

$d = 2dy - dx;$

$incrEst = 2dy; incrNonEst = 2(dy - dx);$

$x = x_1; y = y_1;$

$writePixel(x, y, color);$

$while (x \leq x_2)$

$\{ if (d < 0)$

$\{ d + incrEst;$

$x++;$

$\}$

$else$

$\{ d + incrNonEst;$

$x++; y++;$

$writePixel(x, y, color);$

$\}$

SCAN CONVERSION DEI POLIGONI

Per effettuare il riempimento del poligono è la seguente: se un punto è all'interno lo si colora. Dunque l'algoritmo consiste nell'ordinamento dei pixel in due categorie: dentro e fuori.

ALGORITMO SCAN-LINE

Funzione per qualsiasi poligono. Fa uso delle linee di scansione del frame buffer per individuare i pixel all'interno del poligono. Per ogni linea vengono individuate le intersezioni con gli spigoli del poligono e pixel tra coppie di intersezioni distanziate accesi.

Se il punto è un massimo o minimo locale restano due altrimenti uno solo (spigoli comuni).

La rasterizzazione procede dal basso verso l'alto e da sinistra a destra.

Ogni scan-line differisce dalla precedente in y di una sola unità.

Il coefficiente angolare dello spigolo è dato da $m = y_b - y_a / x_b - x_a = \frac{1}{x_b - x_a}$

Pertanto una volta calcolata la prima intersezione le successive si ottengono incrementando y di 1 e x di $1/m$. L'algoritmo utilizza una tabella di spigoli, ~~ovvero una tabella~~ ^{o lista} di puntatori avente tante entrate quante le linee.

La s -esima posizione della Et , se si riporta alla lista degli spigoli con ordinata min_y .

All'interno di ogni posizione, ~~gli spigoli attivi~~ ^(AET) gli spigoli sono ordinati per x_{min} .

L' Et è affiancata dalle tabelle degli spigoli attivi (AET) (spigoli toccati dalla scanline corrente).

PASSI ALGORITMO

- 1) imposta la scan-line corrente alla più piccola ordinata y per cui corrispondente posizione in Et non sia nulla
- 2) inizializza AET alle stampe vuote
- 3) Ripete i seguenti passi fino a che Et e AET non siano vuote
 - sposta in AET gli spigoli relativi all'ordinata y
 - accende i pixel corrispondenti agli intervalli di ascisse di posizione dispari pari
 - rimuovi da AET gli spigoli per cui $y = y_{max}$
 - incrementa y di 1; in caso di spigoli non verticali e incrementa x di $1/m$

CLIPPING

Si occupa dell'eliminazione delle parti degli oggetti al di fuori del volume di viste.
Le primitive che cadono entro il volume di visualizzazione sono accettate mentre ^{dim} quelle che cadono solo parzialmente devono essere tagliate per eliminare la parte eccedente.
Può essere eseguito prima oppure nel corso delle scan-conversion. E' conveniente effettuarlo prima delle scan-conversion in modo da far eseguire le scan conversion solo con la ^{parte} delle primitive che fanno parte del volume di visualizzazione.
Il clipping può essere determinato calcolando le intersezioni tra le rette dei segmenti e i bordi delle finestre di clipping. Il calcolo richiede calcoli in virgola mobile.

ALGORITMO COHEN-SUTHERLAND

La maggior parte di moltiplicazioni e divisioni in virgola mobile vengono sostituite da sottrazioni e operazioni su bit. I lati delle finestre di clipping vengono estesi all'infinito dividendo il piano in 9 regioni a cui ognuna è assegnato un codice binario

($b_0 b_1 b_2 b_3$) chiamato **outcode**

Se $b_0 = 1$ $y > y_{max}$	Se $b_0 = 0$ $y \leq y_{max}$
Se $b_1 = 1$ $y < y_{min}$	Se $b_1 = 0$ $y \geq y_{min}$
Se $b_2 = 1$ $x > x_{max}$	Se $b_2 = 0$ $x \leq x_{max}$
Se $b_3 = 1$ $x < x_{min}$	Se $b_3 = 0$ $x \geq x_{min}$

CASO 1 $01 = 02 = 0000 \Rightarrow$ segmento interno

CASO 2 $01 \& 02 \neq 0000 \Rightarrow$ segmento esterno

CASO 3 $01 \neq 0000$ e $02 = 0000$ o viceversa

Si calcola l'intersezione tra 01 e il bordo delle finestre.

Si aggiorna 01 con l'intersezione

CASO 4 01 e $02 \neq 0000$

gli estremi sono entrambi esterni ma la linea non viene rifiutata poiché potrebbe intersecare la finestra.

Come si nota le intersezioni vengono calcolate solo se necessario.

ANTI_ALIASING

Si verifica quando si passa dalla rappresentazione continua a quella discreta. **ALIASING**

Le tecniche di riduzione dell'aliasing sono dette di anti-aliasing. Per esempio gli errori avvengono per 3 problemi: 1. segmenti tra loro diversi potrebbero essere approssimati dallo stesso pattern di pixel, 2. i pixel sono fissati su una griglia uniforme, 3. i pixel hanno dimensione e forma fissate.

Un segmento per esempio non dovrebbe accendere un solo pixel con colore nero ma più pixel per riga o colonna con intensità opportune. così da rendere più reale l'oggetto che si vuole rappresentare. **METODO UAS** ipotizza una risoluzione del frame buffer doppia a quella reale suddividendo ogni pixel in almeno 4 subpixel

RIMOZIONE DELLE SUPERFICI NASCOSTE

Prima della scan conversion bisogna determinare se l'oggetto è visibile all'osservatore. Gli algoritmi per la rimozione delle superfici nascoste si dividono in:

OBJECT SPACE

Dati due poligoni A e B si possono presentare 4 casi:
1) A oscura B 2) B oscura A 3) A e B sono completamente visibili 4) A e B si oscurano parzialmente l'un l'altro; bisogna calcolare le parti visibili di ciascuno.
Si prende uno dei K poligoni e lo si confronta con tutti i restanti determinando la parte visibile. A ogni passo si confronta il poligono in esame con tutti i rimanenti fino a quando rimangono solo due poligoni. La complessità di questo approccio è $O(K^2)$ ed è consigliabile solo se gli oggetti sono pochi.

IMAGE SPACE

Per ogni pixel si considera un raggio che parte dal centro di proiezione e passa da quel pixel; il raggio è intersecato con ciascun dei piani determinati da K poligoni. Si determina l'intersezione più vicina al centro di proiezione e si colora quel pixel. Operazione fondamentale è dunque il calcolo delle intersezioni dei raggi di proiezione con i poligoni. Per un display $n \times m$ l'operazione viene effettuata $n \times m \times K$ con complessità $O(K)$. Per ridurre il carico di lavoro si può eliminare la parte posteriore del poligono con il comando `glCullFace()`.

DEPTH-SORT

È un algoritmo object-space variante di un algoritmo più semplice: Algoritmo del Pittore. Consiste nel dipingere i poligoni dal più lontano al più vicino in modo che i secondi coprano i primi. Dunque alla base dell'algoritmo vi è l'ordinamento dei poligoni. Se la profondità minima è maggiore alla profondità massima del poligono situato sul retro possiamo dipingere il poligono posteriore a quello in profondità. Gli altri poligoni non possono essere dipinti solo sull'estensione lungo z ma anche facendo test sulle coordinate x e y: se non vi è sovrapposizione in nessuno dei due sicuramente nessuno dei due poligoni può oscurare l'altro. Situazioni complesse su questo algoritmo si hanno quando 3 o più algoritmi si sovrappongono ciclicamente fra loro oppure quando uno penetra dentro un altro. In assenza di questi casi l'algoritmo ha complessità $O(K \log K)$.

Z-BUFFER

È un algoritmo molto semplice che si basa l'aveva di fianco l'algoritmo di Scan-Conversion. Necessita di una memoria aggiuntiva per la memorizzazione delle profondità relative ai pixel dette **z-buffer**. Nonostante sia un algoritmo di tipo Image-Space i calcoli effettuati dall'algoritmo non sono relativi ai singoli pixel ma ai poligoni presenti nella scena (**approccio ibrido**).

Poiché si procede poligono per poligono, non si dispone di tutte le informazioni sugli altri poligoni. Tuttavia si possono memorizzare e aggiornare le informazioni relative alla profondità che si rendono disponibili via via nel corso della scan-conversion. Ogni elemento è inizializzato al valore delle distanze massime dal centro di proiezione. Il frame buffer è inizializzato al colore di sfondo.

La computazione avviene nel seguente modo:

- 1) Si effettua la scan conversion poligono per poligono
- 2) Per ciascun punto si calcola la distanza dal centro di proiezione e lo si confronta con quello memorizzato nello z-buffer in corrispondenza del pixel.
- 3) Se la distanza è maggiore della distanza nello z-buffer - significa che abbiamo già un poligono più vicino all'osservatore, dunque non si ~~aggiorna~~ ~~il pixel~~ aggiorna lo z-buffer.
- 4) Se la distanza è minore significa che il poligono è più vicino e dunque si aggiorna lo z-buffer.

INTERAZIONI LUCE-MATERIA

Un modello d'illuminazione è una formulazione matematica dell'equazione del trasporto dell'energia luminosa. Esso descrive come un punto è illuminato. *rendering fotorealistico*
lighting (radiazione luminosa incidente) **shading** (colore risultante)

EQUAZIONE RADIANTZA $L_o(x, \vec{w}_o) = L_e(x, \vec{w}_o) + \int_{\Omega} L_i(x, \vec{w}_i) (\vec{w}_i \cdot \vec{n}) f_r(x, \vec{w}_i, \vec{w}_o) d\vec{w}_i$

Da una posizione x e $\vec{w}_o$ date, l'ammontare di luce uscente L_o è la somma delle luci emesse L_e e della luce riflessa L_r . L_e è la somma dei raggi incidenti, L_i proviene da tutte le direzioni $\vec{w}_i$ per $\cos(\theta)$ per la formula di riflessione della superficie f_r

MODELLO DI PHONG

L'equazione della radianza è troppo onerosa e dunque Phong ha creato un modello semplificato:
1) le sorgenti sono puntiformi 2) non si tiene conto delle inter-riflessioni
3) il calcolo dell'equazione è svolto in maniera locale, senza tener conto di occlusioni
4) la funzione di riflessione f_r è approssimata con due costanti che caratterizzano il materiale
5) l'unico fenomeno fisico è la riflessione (speculare e diffusa)

EQUAZIONE DELLA LUCE AMBIENTE NEL MODELLO DI PHONG $L_{amb} = L_a * K_a$

RIFLESSIONE SPECULARE

L'intensità luminosa dipende dalla posizione dell'osservatore in quanto l'angolo d'incidenza che la luce forma con il normale della superficie è $\theta_i = \theta_r$ come massima riflessione in $\theta = 0$ seguendo la legge $(\cos \theta)^n$ dove n è lo specular reflection point [1, 128]
 $L_{spec} = L_p * K_s * (\cos \theta)^n = L_p * K_s (R \cdot V)^n$

RIFLESSIONE DIFFUSA (in tutte le direzioni)

I raggi luminosi vengono diffusi uniformemente in tutte le direzioni: con intensità luminosa proporzionale al coseno dell'angolo compreso tra la direzione ^{del raggio} e la normale della superficie
 $L_{diff} = L_p * K_d * \cos(\theta) = L_p * K_d (\vec{N} \cdot \vec{L})$ $\vec{N}$ = normale superficie K_d = costante di diffusione
 $\vec{L}$ = direzione luce

FORMULAZIONE COMPLETA MODELLO DI PHONG

$$L = \underbrace{L_a * K_a}_{\text{Luca ambiente}} + \sum_p \left(\underbrace{L_p (K_d * \vec{N} \cdot \vec{L})}_{\text{diffusa}} + \underbrace{K_s (R \cdot V)^n}_{\text{speculare}} \right)$$

TECNICHE DI SHADING

Il colore finale da attribuire al pixel avviene tramite tecniche di SHADING. La più precisa consiste nel calcolare l'equazione del modello di PHONG per ogni pixel ma ovviamente per efficienza questo non è possibile e si ricorre ad approssimazioni.

Lo shading costante (**flat shading**) è il modello più semplice e consiste nel calcolare il modello di PHONG per ogni primitiva estendo poi il risultato su tutte le superfici. Il **Gouraud shading** calcola l'equazione d'illuminazione per ogni vertice di ogni primitiva. Il **Phong shading** garantisce risultati migliori del precedente nei casi di superficie ad alto coefficiente speculare e l'equazione viene calcolata in tutti i punti della primitiva.

RAY-TRACING

Si basa sull'osservazione che solo un sottoinsieme di raggi giunge all'osservatore, anche sotto forma di riflessione e rifrazione. L'idea alla base è quella di seguire la traiettoria dei raggi che colpiscono l'osservatore, dall'osservatore alla sorgente. Data la posizione dell'osservatore, a ogni pixel corrisponde un raggio primario (view ray) dall'osservatore alla scena. Ciascun raggio può perdersi all'infinito o intersecare oggetti presenti nella scena. Nel primo caso al pixel corrispondenti ai raggi andati a vuoto si assegna colore di sfondo. Nel secondo caso si calcola l'intersezione con l'oggetto più vicino e si valuta se è in zona ombra. Per valutare ciò si traccia un raggio ombra verso ogni sorgente della scena. Se il raggio ombra interseca una superficie ^{tepe} ~~esterna~~ allora ricade in zona ombra. I raggi primari e quelli ombra permettono di modellare le ombre portate. Se un raggio primario interseca un punto c di una superficie S caratterizzata da riflessività speculare è necessario tracciare un altro raggio detto **raggio di riflessione speculare**. Se un raggio primario interseca un punto c in una superficie F caratterizzata da un comportamento rifrattivo o semi trasparente è necessario tracciare un ~~altro~~ raggio detto **raggio di trasparenza**. Dunque per ogni intersezione dovuta a un raggio primario ^{interagisce con l'oggetto} il ray-tracing genera una serie di **raggi secondari: ombre, speculari, trasparenza**. Per ogni raggio primario il ray-tracing genera un albero di raggi. I criteri di terminazione del processo di ricorsione sono: 1) profondità massima di ricorsione pari al massimo 3. 2) soglie minima di contributo: non si ~~tracciano~~ tracciano raggi il cui contributo sia inferiore a una soglia prestabilita.

EQUAZIONE ILLUMINAZIONE:
$$L = k_e * L_a + k_d * L_d + k_s * L_s + k_{sro} * L_{sro} + k_{st} * L_{st}$$

componente ambiente riflessione radiazione sorgente riflessione speculare sorgente riflessione speculare altri fonti componente trasmissioni

costanti materiali k_{sro}

RADIOSITY

A differenza del ray-tracing il radiosity permette di ottenere un'illuminazione morbida valutando (esclusivamente) la componente diffusa; gli altri effetti si ottengono applicando successivamente il RT. Nel metodo radiosity ogni porzione di superficie della scena è essa stessa un illuminante. Definiti i fattori di riflessione r_i per gli elementi che compongono la scena il primo passo dell'algoritmo è individuare i fattori di forma da legare le varie coppie d'elementi. Una volta valutati i fattori di forma l'algoritmo considera per prime i poligoni (patch) che rappresentano le sorgenti luminose.

La luce emessa da questi poligoni è trasmessa successivamente per riflessioni in funzione dei rispettivi fattori di forma. Il processo viene iterato fino al raggiungimento di una condizione d'equilibrio che si valuta confrontando i valori di radianza dei vari poligoni in due passi consecutivi dell'algoritmo.

EQUAZIONE RADIOSITY: $b(i) = e(i) + r(i) * \sum_{s=1}^n b_s * f(i,s)$

PHOTON-TRACING

È un metodo alternativo al radiosity per il calcolo della luce diffusa. È un metodo particellare che calcola come i fotoni emessi dalle sorgenti luminose presenti nella scena si distribuiscono colpendo le superfici e d'essendo da queste riflesse. Il tracciamento della particella termina dopo un numero prestabilito di iterazioni oppure quando la percentuale d'assorbimento supera una soglia prestabilita.

COMPITO

- ① Qual'è il valore iniziale della variabile decisionale d nell'algoritmo di Bresenham?
 $D = 2dy - dx$ con $dy = y_2 - y_1$ e $dx = x_2 - x_1$
- ② Nell'algoritmo di Bresenham quali valori può assumere la variabile d ?
Esso può assumere due valori al passo i ^{rispetto} al passo $i-1$.
Se $d < 0$ allora $d_i = d_{i-1} + 2dy$ incremento a est $(i+1, y)$
se $d > 0$ allora $d_i = d_{i-1} + 2(dy - dx)$ incremento a nord-est $(i+1, y+1)$
- ③ Nell'Algoritmo Scan-line qual'è l'incremento di x e y ?
 $y+1; x+\frac{1}{m}$
- ④ Descrivere l'equazione d'illuminazione ambientale del modello di Phong e dire che effetto ambientale genera:
 $L_{amb} = L_a * K_a$
 L_a è la quantità di luce emessa mentre K_a è la costante che approssima la funzione di riflessione delle superfici.
~~Sviluppa il fenomeno dell'INTERREFLESSIONE~~
- ⑤ Descrivere l'equazione di riflessione diffusa del modello di Phong e dire quale fenomeno ambientale sviluppa
 $L_{diff} = L_r * K_d (\vec{N} * \vec{L})$
Con L_r è la quantità di luce riflessa, K_d la costante che approssima la riflessione della superficie, $\vec{N}$ la normale alla superficie $\vec{L}$ la direzione del raggio.
Sviluppa il fenomeno del Bounce.
- ⑥ Descrivere la gestione delle ombre nel modello di Phong
Phong non gestisce le ombre portate poiché gestisce la scena a livello locale senza tener conto di possibili occlusioni tra cui le ombre.
- ⑦ Qual'è la differenza tra il modello di Phong e il Ray-Tracing?
Il primo è locale mentre il secondo è globale. Infatti in Phong non si gestiscono fenomeni particolari come ombre ecc... mentre il Ray-Tracing invece da ogni raggio primario che interseca una superficie vengono lanciati altri due raggi che verificano se si tratta di zona d'ombra e se riflette specularmente.
- 8) Quanto un segmento è completamente esterno? 01 & 02! 0000